

דוח מסכם פרויקט רשתות: חלק 2 - יישום וניתוח תעבורה

שמות הסטודנטים: נועה צדוק 214396434, לירן שטרנברג 211895578 ונעם רחמים 212551121

תאריך: 07.01.2026

[קישור ל-GITHUB](#)

1. הסבר כללי על המערכת ומבנה הקוד

המערכת שפותחה היא אפליקציית צ'אט המדמה את אפליקציית WhatsApp, ופועלת בארכיטקטורת Client-Server (שרת-לקוח) על גבי פרוטוקול TCP. המערכת מאפשרת למספר משתמשים להתחבר במקביל לשרת מרכזי, לקבל רשימת משתמשים מחוברים בזמן אמת, ולנהל שיחות פרטיות עם ממשק גרפי מתקדם.

רכיבי המערכת:

א. צד השרת - server.py

השרת מהווה את "המוח" של המערכת ואחראי על ניהול החיבורים והעברת ההודעות.

תקשורת (Sockets) – השרת מאזין בפורט הקבוע 5566 ובכתובת 0.0.0.0 (מקבל תקשורת מכל ממשק רשת).

ריבוי תהליכים (Multi-Threading): כדי לעמוד בדרישה לתמוך ב-5 לקוחות לפחות בו-זמנית, השרת יוצר Thread (תהליכון) ייעודי לכל לקוח שמתחבר באמצעות הפונקציה handle_client. כך השרת פנוי לקבל חיבורים נוספים כל הזמן.

ניהול משתמשים וסנכרון (Lock):

המערכת מנהלת מילון clients הממפה בין שם המשתמש ל-Socket שלו.

מכיוון שמשאב זה משותף לכל התהליכים, נעשה שימוש ב-threading.Lock. כל גישה למילון (הוספה, מחיקה, או בדיקת יעד) נעשית בתוך בלוק with lock, מה שמונע Race Conditions - התנגשות בין תהליכים המנסים לגשת לאותו משאב בו-זמנית - וקריסת נתונים.

פרוטוקול האפליקציה (Application Layer Protocol):

- התחברות: הלקוח שולח שם משתמש -> השרת בודק כפילות -> מחזיר APPROVE או REFUSED.
- עדכון רשימה: בעת שינוי בסטטוס משתמשים, השרת שולח הודעת LIST[user1,user2]... לכולם.
- העברת הודעה: הודעה נשלחת מהלקוח בפורמט target|message. השרת מנתב אותה ליעד בפורמט MSG|sender|content.

ב. צד הלקוח (client_gui.py)

הלקוח הוא ממשק המשתמש (GUI) המאפשר אינטראקציה נוחה.

ממשק גרפי (Tkinter): הממשק עוצב בסגנון "Dark Mode" המזכיר את WhatsApp Web, וכולל בועות הודעה מעוצבות, רשימת אנשי קשר דינמית ומונה הודעות שלא נקראו.

תקשורת אסינכרונית: כדי שהממשק הגרפי לא "יקפא" (Freeze) בזמן ההמתנה להודעה, הלקוח מריץ Thread נפרד (receive_messages) שמאזין באופן רציף לתעבורה מהשרת ומעדכן את המסך בזמן אמת, בעוד ה-Main Loop של ה-GUI מטפל בלחיצות המשתמש.

2. הוראות התקנה והרצה

כדי להפעיל את המערכת בסביבת פיתוח מקומית:

דרישות קדם: וודא שמותקנת סביבת Python 3.x.

הפעלת השרת:

- פתח מסוף (Terminal/CMD).
- הרץ את הפקודה: python server.py
- פלט צפוי: Server running on 0.0.0.0:5566....

הפעלת הלקוחות:

- פתח מסוף חדש והרץ: `python client_gui.py`
- בחלון הקופץ, הזן שם משתמש (למשל noa).
- פתח מסופים נוספים והתחבר עם שמות שונים (למשל lea, abigail).

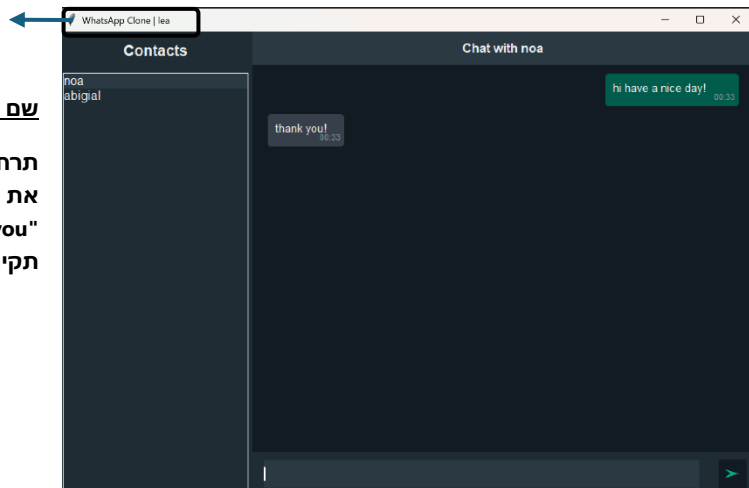
3. דוגמאות קלט ופלט (GUI)

להלן תצוגת המערכת בזמן פעולה. ניתן לראות את רשימת אנשי הקשר בצד שמאל, ואת חלון הציאט עם בועות ההודעה (ירוק עבורי, אפור עבור הצד השני).

WhatsApp Clone | lea

שם השתמש המחובר לאותו חלון

תרחיש שיחה בין lea ל-noa. ניתן לראות את ההודעות "hi have a nice day" ו-"thank you" מוצגות בצורה ויזואלית תקינה



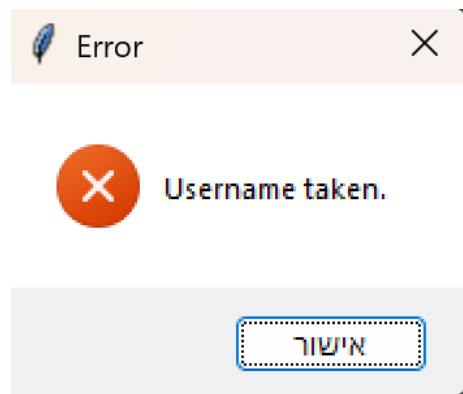
תהליך שליחת הודעה:

קלט: המשתמש בוחר איש קשר ומקליד הודעה בתיבה התחתונה.

עיבוד: הלקוח אורז את ההודעה לפורמט `target|message` ושולח לשרת.

פלט: ההודעה מופיעה מיד במסך השולח כבועה ירוקה, ומגיעה לנמען כבועה אפורה.

במקרה והמשתמש רוצה להתחבר עם אותו שם המשתמש לא יהיה ניתן ע"פ דרישות הפרויקט.



מה קורה בצד השרת?

with lock: השרת נועל את הגישה למשתנים המשותפים כדי למנוע מצב ששני אנשים מתחברים באותה מאית שנייה עם אותו שם.

if username in clients: זו הבדיקה עצמה. המשתנה `clients` הוא מילון שמחזיק את כל המחוברים. השרת בודק האם השם שהוקלד כבר קיים כמפתח במילון הזה.

client_socket.send("REFUSED".encode()): אם השם קיים, השרת שולח בחזרה ללקוח את המחרוזת "REFUSED".

client_socket.close(): השרת מנתק את החיבור מיד.

מה קורה בצד הלקוח?

כשהקוד מקבל את הסירוב מהשרת הוא מקפיץ את הודעת השגיאה :

Username taken.

```
try:
    client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    client.connect((SERVER_IP, PORT))
    client.send(input_user.encode())

    resp = client.recv(1024).decode()
    if resp == "APPROVE":
        username = input_user
    else:
        messagebox.showerror("Error", "Username taken.")
    client.close()
```

4. ניתוח תעבורה של היישום (Wireshark)

בפרק זה ננתח את התעבורה שנלכדה באמצעות Wireshark בממשק Adapter for loopback traffic capture (כתובת 127.0.0.1).

לכידת תעבורת TCP בין פורט 5566 לפורטים אקראיים של הלקוח

No	Time	Source	Destination	Protocol	Length	Info
1	21.528895	127.0.0.1	127.0.0.1	TCP	56	Seq=8 Win=65535 Len=0 MSS=65535 SACK_PERM [SYN] 5566 → 58020 56
2	21.528955	127.0.0.1	127.0.0.1	TCP	56	Ack=1 Win=65535 Len=0 MSS=65535 SACK_PERM [SYN, ACK] 58020 → 5566 56
3	21.530976	127.0.0.1	127.0.0.1	TCP	44	Seq=1 Ack=1 Win=65280 Len=0 [PSH, ACK] 5566 → 58020 44
4	21.530981	127.0.0.1	127.0.0.1	TCP	44	Seq=2 Ack=4 Win=65280 Len=0 [ACK] 58020 → 5566 44
5	21.530982	127.0.0.1	127.0.0.1	TCP	44	Seq=3 Ack=4 Win=65280 Len=0 [PSH, ACK] 58020 → 5566 44
6	21.531080	127.0.0.1	127.0.0.1	TCP	44	Seq=4 Ack=4 Win=65280 Len=0 [PSH, ACK] 58020 → 5566 44
7	21.531400	127.0.0.1	127.0.0.1	TCP	44	Seq=32 Ack=4 Win=65280 Len=0 [PSH, ACK] 58010 → 5566 44
8	21.531528	127.0.0.1	127.0.0.1	TCP	44	Seq=4 Ack=52 Win=65280 Len=0 [ACK] 5566 → 58010 44
9	21.531566	127.0.0.1	127.0.0.1	TCP	44	Seq=24 Ack=8 Win=65280 Len=20 [PSH, ACK] 58013 → 5566 44
10	21.531586	127.0.0.1	127.0.0.1	TCP	44	Seq=24 Ack=8 Win=65280 Len=20 [PSH, ACK] 58013 → 5566 44
11	21.531604	127.0.0.1	127.0.0.1	TCP	44	Seq=24 Ack=8 Win=65280 Len=20 [PSH, ACK] 58013 → 5566 44
12	21.531813	127.0.0.1	127.0.0.1	TCP	44	Seq=8 Ack=4 Win=65280 Len=20 [PSH, ACK] 58020 → 5566 44
13	21.531825	127.0.0.1	127.0.0.1	TCP	44	Seq=4 Ack=28 Win=65280 Len=0 [ACK] 5566 → 58020 44
14	21.547556	127.0.0.1	127.0.0.1	TCP	44	Seq=4 Ack=28 Win=65280 Len=0 [PSH, ACK] 5566 → 58020 44
15	21.547724	127.0.0.1	127.0.0.1	TCP	44	Seq=70 Ack=77 Win=65280 Len=0 [ACK] 58070 → 5566 44
16	21.547797	127.0.0.1	127.0.0.1	TCP	44	Seq=52 Ack=4 Win=65280 Len=27 [PSH, ACK] 58010 → 5566 72
17	21.547943	127.0.0.1	127.0.0.1	TCP	44	Seq=4 Ack=79 Win=65280 Len=0 [ACK] 5566 → 58010 44
18	42.533285	127.0.0.1	127.0.0.1	TCP	44	Seq=4 Ack=79 Win=65280 Len=34 [PSH, ACK] 5566 → 58010 58
19	42.533448	127.0.0.1	127.0.0.1	TCP	44	Seq=79 Ack=18 Win=65280 Len=0 [ACK] 58010 → 5566 44
20	42.533287	127.0.0.1	127.0.0.1	TCP	44	Seq=28 Ack=27 Win=65280 Len=18 [PSH, ACK] 58020 → 5566 62
21	42.532868	127.0.0.1	127.0.0.1	TCP	44	Seq=27 Ack=66 Win=65280 Len=0 [ACK] 5566 → 58020 44
22	57.737511	127.0.0.1	127.0.0.1	TCP	44	Seq=18 Ack=79 Win=65280 Len=25 [PSH, ACK] 5566 → 58010 76
23	57.737556	127.0.0.1	127.0.0.1	TCP	44	Seq=79 Ack=15 Win=65280 Len=0 [ACK] 58010 → 5566 44
24	57.737702	127.0.0.1	127.0.0.1	TCP	44	Seq=44 Ack=8 Win=65280 Len=35 [PSH, ACK] 58015 → 5566 79
25	57.737708	127.0.0.1	127.0.0.1	TCP	44	Seq=48 Ack=79 Win=65280 Len=0 [ACK] 5566 → 58015 44
26	66.678892	127.0.0.1	127.0.0.1	TCP	44	Seq=8 Ack=79 Win=65280 Len=14 [PSH, ACK] 5566 → 58015 58
27	66.678919	127.0.0.1	127.0.0.1	TCP	44	Seq=79 Ack=22 Win=65280 Len=0 [ACK] 58015 → 5566 44
28	66.678969	127.0.0.1	127.0.0.1	TCP	44	Seq=79 Ack=55 Win=65280 Len=22 [PSH, ACK] 58010 → 5566 66
29	66.678951	127.0.0.1	127.0.0.1	TCP	44	Seq=53 Ack=82 Win=65280 Len=0 [ACK] 5566 → 58010 44
30	115.883490	127.0.0.1	127.0.0.1	TCP	44	Seq=8 Win=65535 Len=0 MSS=65535 SACK_PERM [SYN] 445 → 5566 76
31	115.883488	127.0.0.1	127.0.0.1	TCP	44	Ack=1 Win=65535 Len=0 MSS=65535 SACK_PERM [SYN, ACK] 5566 → 445 76

א. שכבת הרשת (IP - Network Layer)

כתובות: התקשורת כולה מתבצעת בין 127.0.0.1 ל-127.0.0.1.Dst.

משמעות: שימוש ב-Loopback מעיד שהתהליכים רצים על אותה מכונה פיזית. המידע אינו יוצא לרשת החיצונית.

ב. שכבת התעבורה (TCP - Transport Layer)

פרוטוקול TCP נבחר בשל אמינותו (Reliability) והבטחת סדר ההודעות, הקריטיים לאפליקציית צ'אט.

1. הקמת הקשר (Three-Way Handshake):

- ניתן לראות את תהליך יצירת הקשר בשורות הראשונות של הלוג (שורות 12-14 בתמונה):
- SYN: הלקוח שולח בקשת התחברות לפורט 5566.
- SYN,ACK: השרת מאשר ומסנכרן
- ACK: הלקוח מאשר. החיבור במצב ESTABLISHED.

2. העברת נתונים (Data Transfer): ההבדל בין LEN=0 ל-LEN>0:

בניתוח הלוג ניתן להבחין בשני סוגים עיקריים של חבילות לאחר החיבור:

א. חבילות מידע (Push):

- מסומנות בדגלים [PSH, ACK].
- אורך המידע (Len) גדול מ-0.
- משמעות: אלו החבילות שמכילות את המידע האמיתי של הצ'אט (שמות משתמשים, תוכן ההודעות).

- דוגמה מהתמונה: בשורה 15, Len=5. זוהי הודעת ה-Login הראשונה שנשלחה (שם המשתמש היה באורך 5 תווים, למשל "user1"). בשורה 80, Len=11, זוהי הודעת צ'אט.
- ב. חבילות אישור (Acknowledgement):
 - מסומנות בדגל [ACK] בלבד.
 - אורך המידע הוא Len=0.
 - משמעות: פרוטוקול TCP דורש שעל כל מידע שנשלח, הצד השני יחזיר "אישור קבלה".
 - חבילות אלו הן ריקות מתוכן (Payload) ותפקידן רק להגיד "קיבלתי את החבילה הקודמת".
 - לכן ה-Length שלהן הוא 0.

3. חילוץ תוכן ההודעה (Message Payload):

כדי לראות את תוכן הצ'אט הממשי ("ההודעה ששלחתי"), ביצענו פעולת Follow TCP Stream ב-Wireshark.

Seq=4 Ack=79 Win=65280 Len=14 [PSH, ACK] 5566 → 58010 58	TCP	127.0.0.1	127.0.0.1	42.532585 38
Seq=79 Ack=79 Win=0 Len=0 [ACK] 58010 → 5566 58	TCP	127.0.0.1	127.0.0.1	32649 39
Seq=18 Ack=79 Win=0 Len=0 [ACK] 58010 → 5566 59	TCP	127.0.0.1	127.0.0.1	37511 42
Seq=79 Ack=79 Win=0 Len=0 [ACK] 58010 → 5566 60	TCP	127.0.0.1	127.0.0.1	37556 43
Seq=79 Ack=53 Win=0 Len=0 [ACK] 58010 → 5566 61	TCP	127.0.0.1	127.0.0.1	70305 48
Seq=53 Ack=17 Win=0 Len=0 [ACK] 58010 → 5566 62	TCP	127.0.0.1	127.0.0.1	70351 49

4. ניתוק לא תקין (RST Flag)

בשורה 165 (מסומנת באדום בתמונה), נצפה דגל RST, [ACK].

Seq=45 Ack=24 Win=65280 Len=0 [ACK] 53731 → 5566 44	TCP	127.0.0.1	127.0.0.1	88.277657 162
Seq=1 Ack=17 Win=65280 Len=15 [PSH, ACK] 62014 → 5566 59	TCP	127.0.0.1	127.0.0.1	88.278088 163
Seq=17 Ack=16 Win=65280 Len=0 [ACK] 5566 → 62014 44	TCP	127.0.0.1	127.0.0.1	88.278163 164
Seq=24 Ack=45 Win=0 Len=0 [RST, ACK] 5566 → 53731 44	TCP	127.0.0.1	127.0.0.1	90.832225 165
Seq=17 Ack=16 Win=65280 Len=11 [PSH, ACK] 5566 → 62014 55	TCP	127.0.0.1	127.0.0.1	94.501271 166
Seq=16 Ack=28 Win=65280 Len=0 [ACK] 62014 → 5566 44	TCP	127.0.0.1	127.0.0.1	94.501328 167
Seq=16 Ack=28 Win=65280 Len=27 [PSH, ACK] 62014 → 5566 71	TCP	127.0.0.1	127.0.0.1	94.501834 168
Seq=28 Ack=43 Win=65280 Len=0 [ACK] 5566 → 62014 44	TCP	127.0.0.1	127.0.0.1	94.501883 169

דגל Reset נשלח כאשר הקשר נקטע באופן פתאומי ולא מסודר (למשל, סגירת חלון ה-CMD בכוח במקום יציאה מסודרת). השרת מזהה שהלקוח "נעלם" ושולח RST כדי לסגור את ה-Socket ולשחרר משאבים.

5. תיאור שימוש בבינה מלאכותית (AI):

- במהלך הפיתוח נעשה שימוש במודל שפה (LLM) למטרות הבאות:
 - שיפור ממשק המשתמש (GUI): סיוע בכתיבת קוד tkinter ליצירת הופעות המעוגלות ("Bubbles") ועיצוב ה-Dark Mode, דבר שדרש חישוב קואורדינטות מורכב על גבי Canvas.
 - הבנת פרוטוקולים: הסבר על ההבדלים בין דגלי TCP ב-Wireshark (בפרט ההבדל בין חבילות מידע PSH לחבילות אישור ACK עם Len=0).
 - דיבוג (Debugging): איתור בעיה שבה הממשק הגרפי קפא בעת המתנה להודעה, ופתרונה באמצעות שימוש ב-Threading נפרד להאזנה (receive_messages).

דוגמאות לפרומפטים ששימשו אותנו:

- "כיצד ניתן ליצור בועות הודעה (Chat Bubbles) עם פינות עגולות ב-Tkinter כמו ב-WhatsApp?"
- "הסבר לי מדוע אני רואה ב-Wireshark חבילות עם Len=0 ומה התפקיד שלהן בפרוטוקול TCP?"
- "איך למנוע התנגשות בין תהליכים (Race Condition) כאשר שני לקוחות מנסים לעדכן את רשימת המחוברים בשרת במקביל?"